

# Fundamentals of Data Science — Lab Manual

## Lab 0 plus twelve two-hour labs in Google Colab

### Table of Contents

#### What you have, and how labs work

Lab 0 is preparation you do on your own before Week 1. Labs 1 to 12 run one per week.

Every lab follows the same shape, so you always know where you are:

| Time      | Block             | What happens                                            |
|-----------|-------------------|---------------------------------------------------------|
| 0:00–0:10 | <b>Setup</b>      | Open Colab, load the data, run the version block        |
| 0:10–0:25 | <b>Warm-up</b>    | A short task using last week's skill, so it stays fresh |
| 0:25–1:35 | <b>Main work</b>  | Three or four tasks, building on each other             |
| 1:35–1:50 | <b>Checkpoint</b> | Restart and run all; fix what breaks                    |
| 1:50–2:00 | <b>Write-up</b>   | The sentence that interprets your result                |

The last twenty minutes are not optional padding. A notebook that produces numbers but does not run from clean and does not interpret them is not finished work, and from Week 5 it will not pass.

#### Your course materials

| What                                      | Use it for                                                                                  |
|-------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Volume 1 — Main Text</b>               | Read the chapter before each lab                                                            |
| <b>Volume 2 — Reference and Solutions</b> | Keep open while you work: code cookbook, formulas, decision tables, model answers, glossary |
| <b>This Lab Manual</b>                    | The twelve labs, plus Lab 0 preparation                                                     |
| <b>data/ — seven CSV files</b>            | Every lab uses them                                                                         |
| <b>DATA_DICTIONARY.md</b>                 | What each column means                                                                      |

Volume 2 is the one students underuse. Part 2 is a code cookbook organised by task — when you cannot remember how to build a ColumnTransformer or sweep a threshold, it is one lookup rather than a hunt through the chapters.

## The twelve labs

| Lab | Topic                                       | Read first |
|-----|---------------------------------------------|------------|
| 0   | <b>Preparation — do before Week 1</b>       | —          |
| 1   | Rows, joins, and what is missing            | Ch 1–2     |
| 2   | Summaries by hand, then in code             | Ch 3       |
| 3   | Clean the registry export                   | Ch 4       |
| 4   | Pipelines and leakage                       | Ch 5       |
| 5   | Exploratory analysis that survives scrutiny | Ch 6       |
| 6   | Chart critique and redesign                 | Ch 7       |
| 7   | Probability and simulation                  | Ch 8       |
| 8   | Intervals, resampling, and power            | Ch 9       |
| 9   | The regression workflow                     | Ch 10      |
| 10  | Classification, end to end                  | Ch 11–12   |
| 11  | Clustering with honest uncertainty          | Ch 13      |
| 12  | Time series, text, and a dashboard          | Ch 14–15   |

## The setup cell

Put this at the top of every lab notebook. It never changes.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
DATA = "/content/drive/MyDrive/fds/data/" # change if your folder differs
```

```
import sys
```

```

import numpy as np, pandas as pd, matplotlib.pyplot as plt
import sklearn, scipy

print("Python", sys.version.split()[0], "| pandas", pd.__version__,
      "| sklearn", sklearn.__version__)

students = pd.read_csv(DATA + "students.csv")
enrolments = pd.read_csv(DATA + "enrolments.csv")
activity = pd.read_csv(DATA + "activity.csv")
outcomes = pd.read_csv(DATA + "outcomes.csv")

print(students.shape, enrolments.shape, activity.shape, outcomes.shape)

``{=openxml}
<w:p><w:r><w:br w:type="page"/></w:r></w:p>

```

## Lab 0 — Preparation

**Do this before Week 1.** About 20 minutes, on your own. Nothing here is data science — it is getting the tools working so that Lab 1 is not spent on setup.

If anything below fails, bring it to the first lab and someone will fix it in five minutes. Do not skip Lab 1 because your setup broke.

### Step 1 — A Google account

Go to <https://colab.research.google.com> and sign in. Use your university account if you have one; a personal Gmail works.

Click **New notebook**. In the first cell type `print("hello")` and press **Shift + Enter**. If you see hello, Colab works.

### Step 2 — Put the data in your Drive

Download the seven CSV files from the course page. Then in Google Drive:

1. Create a folder called `fds`
2. Inside it, create a folder called `data`
3. Upload all seven CSVs into `fds/data/`

Your path should end up as `MyDrive/fds/data/students.csv`. If you use a different path, you will need to change one line in every lab, so it is worth matching exactly.

### Step 3 — Run the setup cell

In a new Colab notebook, paste this and run it:

```
from google.colab import drive
drive.mount('/content/drive')

DATA = "/content/drive/MyDrive/fds/data/"

import sys
import numpy as np, pandas as pd, matplotlib.pyplot as plt
import sklearn, scipy

print("Python", sys.version.split()[0], "| pandas", pd.__version__,
      "| sklearn", sklearn.__version__)

students = pd.read_csv(DATA + "students.csv")
enrolments = pd.read_csv(DATA + "enrolments.csv")
activity = pd.read_csv(DATA + "activity.csv")
outcomes = pd.read_csv(DATA + "outcomes.csv")

print(students.shape, enrolments.shape, activity.shape, outcomes.shape)
```

A permission window will appear asking to connect Drive. Accept it. This is the one step that catches people — the window sometimes opens behind your browser.

**You are ready when you see this:**

```
(1200, 7) (4665, 5) (16165, 6) (1200, 7)
```

### Step 4 — Learn the two menu items you will use every week

**Runtime → Restart and run all.** Cells run in the order *you* ran them, not top to bottom. This hides bugs that only appear when someone else opens your notebook. Run this before every submission.

**File → Save a copy in Drive.** Colab wipes its temporary storage after about 90 minutes of inactivity. Your Drive copy is safe; anything else is gone, with no recovery.

### Step 5 — Read this before Lab 1

Read Chapter 1 of Volume 1. It is short. You do not need to understand all of it — you need to arrive knowing what a *unit of observation* is, because Lab 1 is about nothing else.

### Troubleshooting

| Problem                  | Fix                                  |
|--------------------------|--------------------------------------|
| Drive mount does nothing | The permission window is behind your |

| Problem             | Fix                                                       |
|---------------------|-----------------------------------------------------------|
|                     | browser. Look for it.                                     |
| FileNotFoundError   | Your Drive path differs. Check MyDrive/fds/data/ exactly. |
| Shapes do not match | You uploaded the wrong files, or not all seven.           |
| ModuleNotFoundError | Rare in Colab. Try !pip install packagename.              |

## Lab 1 — Rows, joins, and what is missing

Reading: Chapters 1 and 2 · Goal: turn four raw tables into one defensible student-level dataset before computing a result.

Today is intentionally more than a set of short questions. Every result must name its unit of observation, numerator, denominator, and the join that produced it.

### 0:00–0:10 • Setup

Run the setup cell from Lab 0. Confirm the four shapes. Create a new notebook and add one Markdown heading for each task below.

If your setup did not work, sit next to someone whose did and get it fixed now. Do not spend the lab on setup.

### 0:10–0:25 • Task 1 — Write the data contract

Before you count, create a four-row Markdown table with these columns: table, one row means, candidate key, and repetition or time dimension. Fill it for students, enrolments, activity, and outcomes. Then test the candidate keys:

```
checks = {
    "students": (students, ["student_id"]),
    "enrolments": (enrolments, ["student_id", "course_code", "term"]),
    "activity": (activity, ["student_id", "iso_week"]),
    "outcomes": (outcomes, ["student_id"]),
}

for name, (df, key) in checks.items():
    duplicate_keys = df.duplicated(key).sum()
    missing_keys = df[key].isna().any().any()
    print(f"{name:12s} rows={len(df):5d} | "
          f"distinct students={df.student_id.nunique():4d} | "
          f"duplicate candidate keys={duplicate_keys} | "
          f"missing key values={missing_keys}")
```

In Markdown, answer: why is student\_id alone not a key in enrolments or activity? For each table, state the extra dimension that creates repeated student IDs. A key is only a key after you have checked it.

### 0:25–0:45 • Task 2 — Two honest answers from the same join

The Faculty Office asks: “What share of students are part-time?” The Registrar asks: “What share of registrations belong to part-time students?” These sound similar but have different denominators. Compute both, safely:

```
registration_level = enrolments.merge(
    students[["student_id", "study_mode"]],
```

```

    on="student_id", how="left", validate="many_to_one"
)
assert len(registration_level) == len(enrolments)

estimates = pd.Series({
    "share of registrations held by part-time students":
        (registration_level.study_mode == "Part-time").mean(),
    "share of students who are part-time":
        (students.study_mode == "Part-time").mean(),
})
print(estimates.round(3))

```

Make a two-row Markdown answer. For each estimate, give the question, numerator, denominator, and value. Then explain why the registration-level value is not an answer to the Faculty Office question, but is a valid answer to a different operational question.

### 0:45–1:10 • Task 3 — Build one row per student deliberately

Student Support now asks whether part-time students have different average pass rates. A direct groupby on enrolments weights students by how many modules they take. First build a student-level course summary, then join it to the student registry and outcomes table:

```

per_student_course = (
    enrolments.groupby("student_id", as_index=False)
    .agg(
        modules_taken=("course_code", "nunique"),
        mean_final_grade=("final_grade", "mean"),
        pass_rate=("passed", "mean"),
    )
)
assert per_student_course.student_id.is_unique

analysis = (
    students[["student_id", "programme", "study_mode", "entry_score"]]
    .merge(per_student_course, on="student_id", how="left",
          validate="one_to_one", indicator=True)
    .rename(columns={"_merge": "course_match"})
    .merge(outcomes[["student_id", "early_logins",
                    "midterm_satisfaction", "withdrew"]],
          on="student_id", how="left", validate="one_to_one")
)

print(analysis.shape)
print(analysis.course_match.value_counts())
print(analysis.groupby("study_mode").agg(
    students=("student_id", "size"),

```

```
mean_pass_rate=("pass_rate", "mean"),
mean_modules=("modules_taken", "mean"),
).round(3))
```

In Markdown, state what one row of analysis represents, why both joins are one-to-one, and which of the printed quantities answers a question about a typical student. Report the group sizes with the pass-rate comparison.

### 1:10–1:30 • Task 4 — A join that must fail

A safe analysis should stop if the lookup table has duplicate IDs. Create one bad record, confirm that `validate=` blocks it, then observe what an unsafe merge would do:

```
bad_students = pd.concat([students, students.iloc[[0]], ignore_index=True)
```

```
try:
    enrolments.merge(
        bad_students[["student_id", "study_mode"]],
        on="student_id", how="left", validate="many_to_one"
    )
except pd.errors.MergeError as error:
    print("Merge stopped:", error)

unsafe = enrolments.merge(
    bad_students[["student_id", "study_mode"]],
    on="student_id", how="left"
)
print("rows before:", len(enrolments),
      "| rows after unsafe merge:", len(unsafe))
print("duplicated ID:",
      bad_students.loc[bad_students.student_id.duplicated(keep=False),
                      "student_id"].unique())
```

In Markdown, explain why the unsafe merge creates extra rows. Why is `drop_duplicates()` not a defensible automatic repair unless you first inspect which values disagree?

### 1:30–1:40 • Task 5 — Non-response is part of the data

Use the safe student-level information to check whether the satisfaction survey represents the same kinds of students:

```
survey = outcomes.merge(
    students[["student_id", "study_mode", "programme"]],
    on="student_id", how="left", validate="one_to_one"
)
survey["responded"] = survey.midterm_satisfaction.notna()

by_mode = (survey.groupby(["study_mode", "responded"])
```

```
.agg(students=("student_id", "size"),
     mean_early_logins=("early_logins", "mean"))
print(by_mode.round(2))
print(survey.groupby("study_mode").responded.mean().round(3))
```

Write two sentences: one describing the observed response-rate and engagement pattern within study mode, and one saying what you cannot claim about satisfaction for all students. Do not fill missing satisfaction values today.

### 1:40–1:50 • Checkpoint

Runtime – Restart and run all. Fix anything that breaks. Every merge in this notebook must use `validate=`. Then File – Save a copy in Drive.

### 1:50–2:00 • Analyst note

Write a four-sentence note to Student Support: (1) the exact question and unit of observation; (2) one result with its numerator and denominator; (3) the evidence that your table was not expanded by a bad join; and (4) one limitation caused by non-response.

Submit: your notebook. A number without a sentence, unit, and denominator is incomplete. The notebook must run from a clean restart.

Extension. Compare the part-time share at the registration level and the student level with the distribution of `modules_taken` by `study_mode`. Then write one new question for which the registration-level percentage is the correct measure.

## Lab 2 — Summaries by hand, then in code

**Reading:** Chapter 3 · **Goal:** know what a summary discards

### 0:00–0:10 • Setup

### 0:10–0:25 • Warm-up

Name two ways a join can go wrong. One sentence each.

### 0:25–0:50 • Task 1 — By hand, on paper

Take the first eight values of `entry_score`:

```
vals = students.entry_score.head(8).tolist()
print(vals)
```

**On paper**, compute: mean, median, variance, standard deviation, and IQR. Show every step — deviations, squares, the sum.

Then check:

```
s = pd.Series(vals)
print(s.mean(), s.median(), s.var(), s.std())
```

If you disagree with pandas, find out why before moving on. That investigation is the point of the task.

### 0:50–1:05 • Task 2 — The ddof trap

```
import numpy as np
x = np.array(vals)
print("pandas std:", pd.Series(x).std())
print("numpy std: ", np.std(x))
print("numpy ddof=1:", np.std(x, ddof=1))
```

Two libraries, two answers. Explain in Markdown which is which and why. Then compute the size of the discrepancy at  $n = 8$ ,  $n = 100$ , and  $n = 1200$ .

### 1:05–1:35 • Task 3 — When the mean lies

```
g = enrolments.final_grade
print(g.describe().round(2))
g.hist(bins=40)
plt.xlabel("final grade"); plt.show()
```

The histogram has two peaks. Find the grouping variable that explains them:

```
by_course = (enrolments.groupby("course_code").final_grade
              .agg(["count", "mean", "std"]).round(1).sort_values("mean"))
print(by_course)
```

In Markdown: is the cohort mean of 58.8 a useful summary? Say what you would report instead.

### 1:35–1:50 • Checkpoint

Restart and run all. Save.

### 1:50–2:00 • Write-up

Two sentences: one situation where you would report the mean, one where you would report the median, using variables from this dataset.

**Submit:** notebook, including a photo or scan of your handwritten working.

**Extension.** Standardise entry\_score two ways — against the whole cohort, and within programme. Find the student whose rank changes most, and say which method you would defend to an appeals panel.



## Lab 3 — Clean the registry export

**Reading:** Chapter 4 · **Goal:** every cleaning decision justified and counted

Load the damaged file today:

```
dirty = pd.read_csv(DATA + "students_dirty.csv")
```

### 0:00–0:10 • Setup

### 0:10–0:25 • Warm-up

students.csv has 1,200 rows. How many does students\_dirty.csv have? What does the difference tell you before you look at anything else?

### 0:25–0:45 • Task 1 — Inventory before you change anything

Do not fix a single thing yet. Produce a list of what is wrong:

```
print("shape:", dirty.shape)
print("exact duplicate rows:", dirty.duplicated().sum())
print("duplicated student_id:", dirty.student_id.duplicated().sum())
print("programme values:", dirty.programme.nunique())
print(dirty.isna().sum())
print(dirty.dtypes)
```

Write the inventory into a Markdown table: **issue | rows affected**. You will add two more columns as you go.

### 0:45–1:20 • Task 2 — Fix, one issue per cell

Each fix gets its own cell, a comment saying why, and a printed count of rows affected.

Work through: whitespace and case in programme; entry\_score stored as text with %; impossible values in age\_at\_entry; the three date formats in registered\_on.

For the duplicates, be careful. Two rows are identical — safe to drop. Three share a student\_id with **different values**. Look at them:

```
dupe_ids = dirty.student_id[dirty.student_id.duplicated(keep=False)]
print(dirty[dirty.student_id.isin(dupe_ids)].sort_values("student_id"))
```

These are updates, not duplicates. drop\_duplicates() would keep whichever came first, at random. State a rule instead and implement it.

### 1:20–1:35 • Task 3 — Assertions

```
assert clean.student_id.is_unique
assert set(clean.programme) == {"Biology", "Computer Science",
                                "Economics", "Psychology"}
```

```
assert clean.entry_score.dropna().between(0, 100).all()
print("all checks passed —", len(clean), "rows")
```

**Do not impute any missing values today.** Instead, for each column with missing data, write one sentence proposing whether it is MCAR, MAR, or MNAR, and what evidence would settle it.

### 1:35–1:50 • Checkpoint

Restart and run all. Save.

### 1:50–2:00 • Write-up

Complete the cleaning log: **issue | rows affected | action | justification**. Add one decision you would be willing to defend to an auditor.

**Submit:** notebook, cleaning log, cleaned CSV.

**Extension.** One of the three date formats is ambiguous. Construct a date where a wrong parse changes the month, and show what `dayfirst=True` versus `dayfirst=False` does to it.



## Lab 4 — Pipelines and leakage

**Reading:** Chapter 5 • **Goal:** see leakage inflate a score in front of you

*This is the most important lab in the course. If you understand only one thing this term, make it this.*

### 0:00–0:10 • Setup

### 0:10–0:25 • Warm-up

Why does k-means need scaled features when a decision tree does not? Two sentences.

### 0:25–0:50 • Task 1 — Build a pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
```

```
d = outcomes.merge(students, on="student_id")
numeric = ["early_logins", "early_submissions", "entry_score", "age_at_entry"]
categorical = ["programme", "study_mode"]
```

```
pre = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical),
])
model = Pipeline([("pre", pre), ("clf", LogisticRegression(max_iter=1000))])
```

```
X, y = d[numeric + categorical], d.withdrawn.astype(int)
print(cross_val_score(model, X, y, cv=5, scoring="roc_auc").mean().round(3))
```

Record the score.

### 0:50–1:25 • Task 2 — Leak on purpose

Build a second model using **all fourteen weeks** of activity instead of weeks 1–4:

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
from sklearn.pipeline import make_pipeline
```

```
full = (activity.groupby("student_id")
```

```

        .agg(all_logins=("logins", "sum"), all_subs=("submissions", "sum"))
        .reset_index()
d2 = d.merge(full, on="student_id")
y2 = d2.withdrew.astype(int)

for name, cols in [("honest", ["early_logins", "early_submissions", "entry_score"]),
                  ("leaky", ["all_logins", "all_subs", "entry_score"]):
    Xtr, Xte, ytr, yte = train_test_split(d2[cols], y2, test_size=.3,
                                         random_state=0, stratify=y2)
    m = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=1000)).fit(Xtr, ytr)
    print(f'{name:8s} ROC-AUC {roc_auc_score(yte, m.predict_proba(Xte)[:,-1]):.3f}')

```

The leaky model scores higher. Find out why:

```
print(d2.groupby("withdrew").all_logins.mean().round(1))
```

In Markdown, answer in two sentences: why does a withdrawn student have fewer total logins, and would you have this number in week 5?

### 1:25–1:40 • Task 3 — The timing table

For every feature you have used today, fill in a table:

| Feature | Known at prediction time (week 5)? |
|---------|------------------------------------|
|---------|------------------------------------|

Any row answering “no” is leakage.

### 1:40–1:50 • Checkpoint

Restart and run all. Save.

### 1:50–2:00 • Write-up

Two sentences on why the leaky model would pass a code review — the improvement is modest and plausible, not obviously wrong. This is why leakage survives.

**Submit:** notebook with both scores and the timing table.

**Extension.** Add 400 pure-noise columns. Run SelectKBest inside and outside the pipeline, and compare the cross-validated accuracies. Explain the gap.



## Lab 5 — Exploratory analysis that survives scrutiny

**Reading:** Chapter 6 · **Goal:** find a finding, then try to destroy it

### 0:00–0:10 • Setup

### 0:10–0:25 • Warm-up

Name one feature from Lab 4 that leaked, and say at exactly what moment its value becomes known.

### 0:25–0:45 • Task 1 — Questions first

**Before plotting anything**, write three questions in Markdown. Label each descriptive, comparative, or predictive.

You will be marked on whether your analysis answers these, not on how many charts you produce.

### 0:45–1:15 • Task 2 — Compare two groups

Compute the pass rate by study\_mode:

```
m = enrolments.merge(students[["student_id", "study_mode"]], on="student_id")
print(m.groupby("study_mode").passed.mean().round(4))
```

One group does better. Now break it down by course:

```
print(m.groupby(["course_code", "study_mode"]).passed.mean().unstack().round(3))
```

Look carefully at what happened. In Markdown, describe what you see in two sentences.

### 1:15–1:35 • Task 3 — Find the mechanism

Something explains the reversal. Investigate the course mix:

```
hard = ["MA110", "CH110", "ST110"]
m["is_hard"] = m.course_code.isin(hard)
print(m.groupby("study_mode").is_hard.mean().round(3))
```

Explain the reversal in your own words, using these numbers.

**One cell in your course table does not fit the pattern.** Find it, check its sample size, and say whether you would report it as a finding.

### 1:35–1:50 • Checkpoint

Restart and run all. Save.

### 1:50–2:00 • Write-up

Write your finding in three separate parts — **observation, interpretation, limitation**. A finding missing any of the three does not count.

**Submit:** notebook with three questions and at least one full three-part finding.

**Extension.** Test whether the relationship between total logins and mean grade survives stratification by entry-score quartile. Predict first whether it will.



## Lab 6 — Chart critique and redesign

**Reading:** Chapter 7 · **Goal:** see that a chart is an argument

**Bring to this lab:** three misleading charts you have found — news, advertising, a company report. Screenshots are fine.

### 0:00–0:10 • Setup

### 0:10–0:30 • Task 1 — Critique

For each of your three charts, name the specific technique: truncated baseline, area encoding, dual axes, colour-only encoding, cherry-picked range. Estimate how much it exaggerates.

Swap with a neighbour. Can they find a problem you missed?

### 0:30–1:00 • Task 2 — The baseline experiment

```
per_student = (enrolments.groupby("student_id").final_grade.mean()
                .rename("mg").reset_index()
                .merge(students[["student_id", "programme"]], on="student_id"))
means = per_student.groupby("programme").mg.mean().sort_values()
print(means.round(1))
```

Draw the same data twice — once with `ax.set_ylim(58, 60)` and once with `ax.set_ylim(0, 100)`.

Write the headline each chart invites. Then state which is honest, and why the honest one is boring.

### 1:00–1:25 • Task 3 — The greyscale test

Build a scatter of `entry_score` against mean grade, coloured by programme. Save it, then:

```
plt.savefig("chart.png", dpi=150, bbox_inches="tight")
```

```
from PIL import Image
Image.open("chart.png").convert("L").save("chart_grey.png")
```

Open the greyscale version. What is lost? Redesign so the message survives without colour — use shape, faceting, or direct labels.

### 1:25–1:40 • Task 4 — Message titles

Write a message title for each of your figures. For each, state which rung of the claim ladder it sits on and what evidence in the figure supports it. Rewrite any title claiming more than it can support.

### 1:40–1:50 • Checkpoint

Restart and run all. Save.

### 1:50–2:00 • Write-up

One paragraph: which of your three original charts was most misleading, and whether you think it was deliberate.

**Submit:** notebook with three critiques and three redesigns.

**Extension.** Produce two line charts of `platform_daily.csv` that support opposite conclusions, changing only the axis limits and date range.



## Lab 7 — Probability and simulation

**Reading:** Chapter 8 · **Goal:** get conditional probabilities the right way round

### 0:00–0:10 • Setup

### 0:10–0:25 • Warm-up

On paper: a test is 99% accurate for a disease affecting 1 in 10,000. You test positive. Roughly what is the chance you have it? Write your instinct down before calculating anything.

### 0:25–0:55 • Task 1 — The screening table

Build it as a function:

```
def screening(prevalence, sensitivity, specificity, n=10_000):
    ill = prevalence * n
    well = n - ill
    tp = sensitivity * ill
    fn = ill - tp
    tn = specificity * well
    fp = well - tn
    precision = tp / (tp + fp)
    return dict(tp=tp, fp=fp, fn=fn, tn=tn, precision=precision)

for p in [0.01, 0.05, 0.115, 0.25]:
    r = screening(p, 0.90, 0.95)
    print(f'prevalence {p:6.3f} flags {r["tp"]+r["fp"]:7.0f} '
          f'precision {r["precision"]:.3f}')
```

Compare to your instinct from the warm-up. In Markdown, explain the gap.

### 0:55–1:15 • Task 2 — Plot it

Plot precision against prevalence from 0.001 to 0.5. Mark where precision reaches 50%.

In Markdown: what does that point mean for anyone proposing a “majority confidence” threshold?

### 1:15–1:40 • Task 3 — Simulate to check

```
rng = np.random.default_rng(42)
n = 100_000
withdrew = rng.random(n) < 0.01
flag = np.where(withdrew, rng.random(n) < 0.90, rng.random(n) < 0.05)
print("P(withdrew | flagged):", withdrew[flag].mean().round(3))
```

Does it match your table? Then find the real base rate:

```
print("actual withdrawal rate:", outcomes.withdrew.mean().round(3))
```

Re-run the table at that rate and comment on what changed.

### 1:40–1:50 • Checkpoint

Restart and run all. Save.

### 1:50–2:00 • Write-up

Three sentences advising the retention team: at what precision, if any, would an automatic action be justified — and what should happen below that?

**Submit:** notebook with the function, the plot, and the simulation.

**Extension.** Take four students each with a 20% chance of missing a deadline. Compute  $P(\text{at least two miss})$  assuming independence, then simulate it with a shared disruption. Report how much the answer moves.



## Lab 8 — Intervals, resampling, and power

**Reading:** Chapter 9 · **Goal:** report an interval before a p-value

**Also today:** you will swap notebooks with a classmate.

### 0:00–0:10 • Setup

### 0:10–0:30 • Task 1 — The reproduction swap

Send your **Lab 5** notebook to a classmate and open theirs in your own Colab. Try to reproduce their headline finding.

It will probably fail. Write down exactly what broke — usually a file path, a variable defined in a deleted cell, or cells that only work in a particular order.

**This failure is the lesson and costs no marks.** Note the fix and apply it to your own notebook.

### 0:30–1:00 • Task 2 — Interval first

```
from scipy import stats
per_student = (enrolments.groupby("student_id").final_grade.mean()
               .rename("mg").reset_index()
               .merge(students[["student_id", "study_mode"]], on="student_id"))
A = per_student[per_student.study_mode == "Full-time"].mg
B = per_student[per_student.study_mode == "Part-time"].mg

diff = A.mean() - B.mean()
se = np.sqrt(A.var(ddof=1)/len(A) + B.var(ddof=1)/len(B))
print(f'difference {diff:.2f}, 95% CI [{diff-1.96*se:.2f}, {diff+1.96*se:.2f}]')
print(stats.ttest_ind(A, B, equal_var=False))
```

Write the interpretation of the **interval** before you look at the p-value. State what it rules out, not just whether it contains zero.

### 1:00–1:25 • Task 3 — Permutation test from scratch

```
rng = np.random.default_rng(0)
combined = np.concatenate([A.values, B.values])
observed = A.mean() - B.mean()

null = []
for _ in range(10_000):
    shuffled = rng.permutation(combined)
    null.append(shuffled[:len(A)].mean() - shuffled[len(A):].mean())
null = np.array(null)
```

```
p = (np.abs(null) >= abs(observed)).mean()
print("permutation p:", round(p, 4))
plt.hist(null, bins=50); plt.axvline(observed, color="red"); plt.show()
```

Compare to the t-test. What does each assume?

### 1:25–1:40 • Task 4 — Power

The retention team wants to test whether a week-3 email cuts withdrawal from 11.5% to 9.5%.

```
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize
```

```
es = proportion_effectsize(0.115, 0.095)
n = NormalIndPower().solve_power(effect_size=es, alpha=0.05, power=0.80)
print("n per arm:", np.ceil(n))
```

We have 1,200 students. In Markdown: can this study be run? List the options.

### 1:40–1:50 • Checkpoint

Restart and run all — this time it should pass, because you fixed Task 1's problems. Save.

### 1:50–2:00 • Write-up

Three sentences advising the team whether to run the trial, and why.

**Submit:** notebook, plus a note on what broke in your classmate's notebook and what broke in yours.

**Extension.** Draw 200 samples of  $n = 40$ , build a confidence interval for each, and count how many contain the population mean. Plot them.



## Lab 9 — The regression workflow

**Reading:** Chapter 10 • **Goal:** never report a score without a baseline

### 0:00–0:10 • Setup

### 0:10–0:25 • Warm-up

What does a confidence interval of  $[-0.89, 1.74]$  rule out? One sentence.

### 0:25–0:40 • Task 1 — Split once, then leave it alone

```
from sklearn.model_selection import train_test_split
ds = enrolments[enrolments.course_code == "DS100"].merge(students, on="student_id")
X, y = ds[["entry_score"]], ds.final_grade
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)
```

**Do not touch  $X_{te}$  or  $y_{te}$  again until 1:25.** Write that commitment in a Markdown cell.

### 0:40–0:55 • Task 2 — Baseline

```
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error
```

```
base = DummyRegressor(strategy="mean").fit(X_tr, y_tr)
print("baseline MAE:", mean_absolute_error(y_te, base.predict(X_te)).round(2))
```

Wait — you just used the test set. In this one case that is fine, because the baseline is fixed and not selected. Note in Markdown why that is different from tuning against it.

### 0:55–1:25 • Task 3 — Models, cross-validated

Fit linear regression with one predictor, then three, then all available. Cross-validate each on the **training set only**:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
```

```
scores = cross_val_score(LinearRegression(), X_tr, y_tr, cv=5,
                        scoring="neg_mean_absolute_error")
print(f"CV MAE {scores.mean():.2f} ( $\pm$ {scores.std():.2f})")
```

Keep a **model-selection record** in Markdown: every model tried, its CV score, and why you kept or rejected it.

### 1:25–1:40 • Task 4 — Test once, and diagnose

Now evaluate your single chosen model on the test set. Report the difference from CV and explain it.

Plot residuals against fitted values. Is there a pattern?

### 1:40–1:50 • Checkpoint

Restart and run all. Save.

### 1:50–2:00 • Write-up

Three sentences on whether this model should be used for any decision about an individual student. Use the residual spread in your answer.

**Submit:** notebook with the model-selection record.

**Extension.** Compute MAE and RMSE on the test set. Report the ratio, find the three largest residuals, and say whether they are data errors, unusual students, or a missing signal.



## Lab 10 — Classification, end to end

**Reading:** Chapters 11 and 12 · **Goal:** build a classifier, then choose a threshold you could defend to a student

*This is a full lab. Tasks 1 and 2 build the model; Tasks 3 and 4 decide what to do with it. Do not skip Task 4 — it carries the most marks.*

### 0:00–0:10 • Setup

### 0:10–0:20 • Warm-up

Why can a model have 88.6% accuracy and be useless? One sentence.

### 0:20–0:40 • Task 1 — Baselines, and three models

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split, cross_val_score
```

```
F = ["early_logins", "early_submissions", "early_minutes",
     "entry_score", "age_at_entry"]
d = outcomes.merge(students, on="student_id")
X, y = d[F], d.withdrawn.astype(int)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=0, stratify=y)

models = {
    "baseline": DummyClassifier(strategy="most_frequent"),
    "logistic": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
    "knn15": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=15)),
    "tree4": DecisionTreeClassifier(max_depth=4, random_state=0),
}
for name, m in models.items():
    acc = cross_val_score(m, X, y, cv=5, scoring="accuracy").mean()
    auc = cross_val_score(m, X, y, cv=5, scoring="roc_auc").mean()
    ap = cross_val_score(m, X, y, cv=5, scoring="average_precision").mean()
    print(f'{name:10s} acc {acc:.3f} AUC {auc:.3f} PR-AUC {ap:.3f}')
```

The baseline scores high on accuracy and 0.500 on AUC. Explain that in Markdown. Note stratify=y and say why it matters when a class is rare.

### 0:40–1:00 • Task 2 — Read the coefficients

```
clf = models["logistic"].fit(X_tr, y_tr)
lr = clf.named_steps["logisticregression"]
for name, c in zip(F, lr.coef_[0]):
    print(f'{name:20s} {c:+.3f} odds ratio {np.exp(c):.3f}')
print("intercept:", round(lr.intercept_[0], 3))
```

Pick one student from the test set. **By hand, on paper**, compute their predicted probability from the coefficients. Check against `predict_proba`.

This matters beyond the arithmetic: if a student asks why they were flagged, this calculation is the answer. A model you cannot do this for is a model nobody can contest.

### 1:00–1:25 • Task 3 — The accuracy trap and the threshold

```
from sklearn.metrics import (confusion_matrix, precision_score,
                             recall_score, accuracy_score)
p = clf.predict_proba(X_te)[:, 1]

for t in [0.50, 0.30, 0.15, 0.11, 0.05]:
    pred = (p >= t).astype(int)
    print(f't={t:.2f} flagged={pred.sum():3d} acc={accuracy_score(y_te, pred):.3f} '
          f'prec={precision_score(y_te, pred, zero_division=0):.3f} '
          f'rec={recall_score(y_te, pred):.3f}')
```

Print the confusion matrix at 0.50. The model flags almost nobody and scores 88.9%.

Plot precision, recall, and number flagged against threshold on one figure.

**Now the real question.** Three advisers can each contact 40 students. Find the threshold that flags about 120, and report the recall it buys.

### 1:25–1:45 • Task 4 — The fairness audit

```
te = d.loc[y_te.index].copy()
te["flag"], te["y"] = (p >= 0.15).astype(int), y_te.values

for group in ["study_mode", "first_generation"]:
    print(f'\n--- {group} ---')
    for level, g in te.groupby(group):
        if g.y.sum() == 0:
            print(f'{str(level):16s} n={len(g):3d} no positives — cannot compute recall')
            continue
        print(f'{str(level):16s} n={len(g):3d} base={g.y.mean():.3f} '
              f'flagged={g.flag.mean():.3f} rec={recall_score(g.y, g.flag):.3f}')
```

Are flag rates equal across groups? Is recall? When base rates differ, they cannot both be. Say which you would equalise and why, given that a flag triggers an *offer of tutoring*.

**Report  $n$  for every group.** A cell with a handful of positives cannot support a number — say so rather than reporting one to three decimals.

### 1:45–1:50 • Checkpoint

Restart and run all. Save.

### 1:50–2:00 • Write-up

Begin your **threshold memo**: recommended threshold, the error costs justifying it, which fairness criterion you chose, and how a flagged student can find out and contest it. Finish it as homework.

**Submit:** notebook, your handwritten probability calculation, and the one-page memo.

**Extension.** Produce a calibration curve. State whether the probabilities can be used directly for triage, or only for ranking.

## Lab 11 — Clustering with honest uncertainty

**Reading:** Chapter 13 · **Goal:** find out whether your clusters are real

0:00–0:10 · Setup

0:10–0:25 · Warm-up

Why is equal accuracy across two groups almost no evidence of fairness? One sentence.

0:25–0:45 · Task 1 — Scaling changes the answer

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import adjusted_rand_score
```

```
sub = students.sample(320, random_state=5)
Xr = sub[["age_at_entry", "entry_score"]].values.astype(float)
```

```
lab_raw = KMeans(3, n_init=20, random_state=0).fit_predict(Xr)
lab_scaled = KMeans(3, n_init=20,
random_state=0).fit_predict(StandardScaler().fit_transform(Xr))
print("agreement (ARI):", adjusted_rand_score(lab_raw, lab_scaled).round(3))
```

Plot both. In Markdown: the scaling decision, not the data, produced different groups of people. Say why.

0:45–1:10 · Task 2 — Choosing k, and testing whether k exists

```
from sklearn.metrics import silhouette_score
cols = ["early_logins", "early_submissions", "entry_score", "age_at_entry"]
Z = StandardScaler().fit_transform(d[cols])
```

```
for k in range(2, 9):
    km = KMeans(k, n_init=10, random_state=0).fit(Z)
    print(f"k={k} inertia={km.inertia_:8.1f} silhouette={silhouette_score(Z,
km.labels_):.3f}")
```

Now the test almost nobody runs — shuffle each column independently to destroy any joint structure, and cluster that:

```
rng = np.random.default_rng(0)
Z_null = np.column_stack([rng.permutation(Z[:, j]) for j in range(Z.shape[1])])
for k in [2, 3, 4]:
    km = KMeans(k, n_init=10, random_state=0).fit(Z_null)
    print(f"null k={k} silhouette={silhouette_score(Z_null, km.labels_):.3f}")
```

Compare. Does your real silhouette beat the null?

### 1:10–1:35 • Task 3 — Profile and name

Fit  $k = 3$  and profile the clusters. Give each a **neutral name** describing observed features only — no dispositions, no predictions.

Then check what your clusters correlate with:

```
d2 = d.copy()
d2["cluster"] = KMeans(3, n_init=20, random_state=0).fit_predict(Z)
print(d2.groupby("cluster").withdrew.mean().round(3))
print(pd.crosstab(d2.cluster, d2.study_mode, normalize="index").round(3))
```

**One of your clusters is not a discovery.** Identify it and say what it actually is.

### 1:35–1:50 • Checkpoint

Restart and run all. Save.

### 1:50–2:00 • Write-up

For each cluster: a neutral name, one proposed intervention, and one risk of getting it wrong.

**Submit:** notebook plus a one-page segment brief.

**Extension.** Refit  $k = 3$  across ten random seeds and report pairwise ARI. If the solutions disagree, what does that mean for naming them?



## Lab 12 — Time series, text, and a dashboard

**Reading:** Chapters 14 and 15 · **Goal:** beat a baseline that is harder than it looks

This lab is fuller than the others. Do the time-series half properly; the text half is shorter by design.

### 0:00–0:05 • Setup

```
daily = pd.read_csv(DATA + "platform_daily.csv",
parse_dates=["date"]).sort_values("date")
feedback = pd.read_csv(DATA + "feedback.csv")
```

### 0:05–0:30 • Task 1 — Baselines you must beat

```
cut = int(len(daily) * 0.8)
train, test = daily.sessions[:cut], daily.sessions[cut:]
```

```
naive = np.full(len(test), train.iloc[-1])
snaive = daily.sessions.shift(7)[cut:].values
mean_f = np.full(len(test), train.mean())
```

```
for name, f in [("naive", naive), ("seasonal naive", snaive), ("mean", mean_f)]:
    print(f'{name:16s} MAE {np.abs(test.values - np.asarray(f)).mean():.1f}')
```

One baseline is four times better than the others. Plot the weekday profile to see why.

### 0:30–1:00 • Task 2 — Beat 88.9

Build lag and rolling features — **carefully**:

```
daily["lag_1"] = daily.sessions.shift(1)
daily["lag_7"] = daily.sessions.shift(7)
daily["roll_7"] = daily.sessions.shift(1).rolling(7).mean()
daily["dow"] = daily.date.dt.dayofweek
```

Note `.shift(1)` **before** `.rolling(7)`. In Markdown, explain what happens without it.

Fit a model with a **chronological** split. Then refit with a random split and compare. Explain the gap in two sentences.

### 1:00–1:30 • Task 3 — Text

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, cross_val_predict
from sklearn.pipeline import make_pipeline
```

```
pipe = make_pipeline(TfidfVectorizer(stop_words="english"),
```

```

        LogisticRegression(max_iter=1000))
print("CV accuracy:", cross_val_score(pipe, feedback.comment, feedback.sentiment,
cv=5).mean().round(3))
print("baseline:", feedback.sentiment.value_counts(normalize=True).max().round(3))

```

Now find the errors and **read them**:

```

pred = cross_val_predict(pipe, feedback.comment, feedback.sentiment, cv=5)
for c in feedback[pred != feedback.sentiment].comment.head(6):
    print("-", c)

```

The mistakes are not random. Classify them into two types and explain why the representation cannot fix either.

### 1:30–1:50 • Task 4 — The dashboard

```

MIN_CELL = 20
summary = (enrolments.merge(students, on="student_id")
            .groupby("course_code")
            .agg(students=("student_id", "nunique"),
                 pass_rate=("passed", "mean")).round(3))
summary.loc[summary.students < MIN_CELL, "pass_rate"] = None
print(summary)

```

Add a study\_mode breakdown and find the cells that must be suppressed. Then show that suppressing **one** cell in a row is not enough, if the row total is published — recover the hidden value by subtraction.

### 1:50–2:00 • Write-up

One sentence per dashboard view: what decision it supports, and what action follows.

**Submit:** notebook covering all four tasks.

**Extension.** Build the dashboard as an actual Streamlit app (!pip install streamlit), with suppression applied in the data layer rather than at display time.



## Quick reference

### The setup cell

At the top of every notebook. See Lab 0, Step 3.

### Before every submission

1. **Runtime** → **Restart and run all**
2. Fix anything that breaks
3. **File** → **Save a copy in Drive**

### What every lab is marked on

- Does the notebook run top to bottom from clean?
- Does every substantive output have a sentence interpreting it?
- Is at least one limitation stated?
- Is the unit of observation correct wherever a count or mean appears?

### Common errors

| Message                                       | Usual cause                                                     |
|-----------------------------------------------|-----------------------------------------------------------------|
| FileNotFoundError                             | Drive not mounted, or DATA path wrong                           |
| KeyError: 'column'                            | Column renamed, or you are using the wrong table                |
| NameError                                     | Cell defining it was deleted or never run — restart and run all |
| ValueError: could not convert string to float | Column is text; check .dtypes                                   |
| Works for you, fails for others               | Cells run out of order                                          |
| Everything vanished                           | Runtime disconnected; re-run from your Drive copy               |

### The two rules

**Restart and run all before submitting.**

**A number without a sentence is not an answer.**